Experiment 5: Dynamic Webpage Using PHP

Subject: Advanced Web Technologies
Experiment No: 05
Topic: Design and Development of a Dynamic Web Application Using PHP (Student Result Management System)
Student Name: Manish Singh
Roll Number: 23/CS/244
Course & Semester: B.Tech CSE, 7th Semester
Date of Submission: October 9, 2026

1. Aim

To design and develop a dynamic, database-driven Student Result Management web application using PHP, demonstrating server-side request processing, database persistence with PDO SQLite, full CRUD operations, dynamic grade and GPA computation, real-time client feedback, and CSV data export without external frameworks.

2. Objectives

1. Understand the client-server request-response lifecycle and server-side script execution in PHP.
2. Build a modular backend architecture cleanly separating:
    - Configuration and Course Rubrics (`config.php`)
    - Database Abstraction and Persistence (`db.php`)
    - RESTful API and Business Logic (`api.php`)
    - Presentation Layer (`index.php`)
    - Client Interactions and Dynamic Validation (`assets/app.js`)
3. Implement complete CRUD functionality (Create, Read, Update, Delete student records).
4. Implement automatic calculation of aggregate marks, percentage, letter grades (A+ to F), GPA (4.0 scale), and academic standing.

5. Provide live client-side arithmetic recalculation in enrollment modals as scores are typed.
6. Build real-time search, multi-tier filtering, modal sheets, and an official printable academic transcript.
7. Support academic record export to CSV format for portability.
8. Ensure robust error handling, boundary validation (0 to 100 marks), duplicate roll number checks, and deletion confirmation dialogs.

3. Theoretical Background

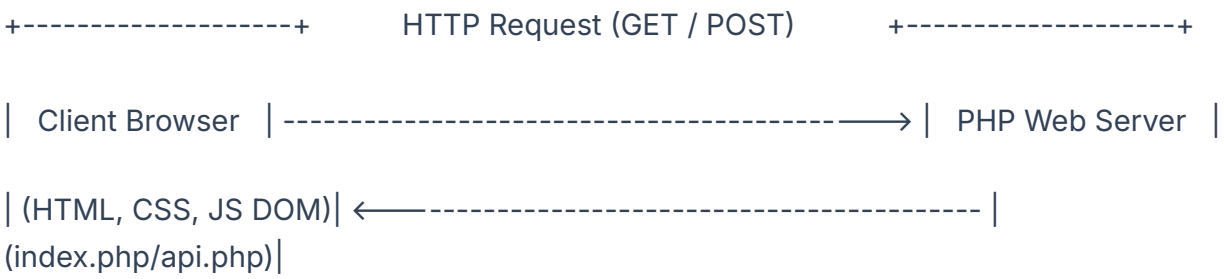
3.1 Static vs. Dynamic Webpages

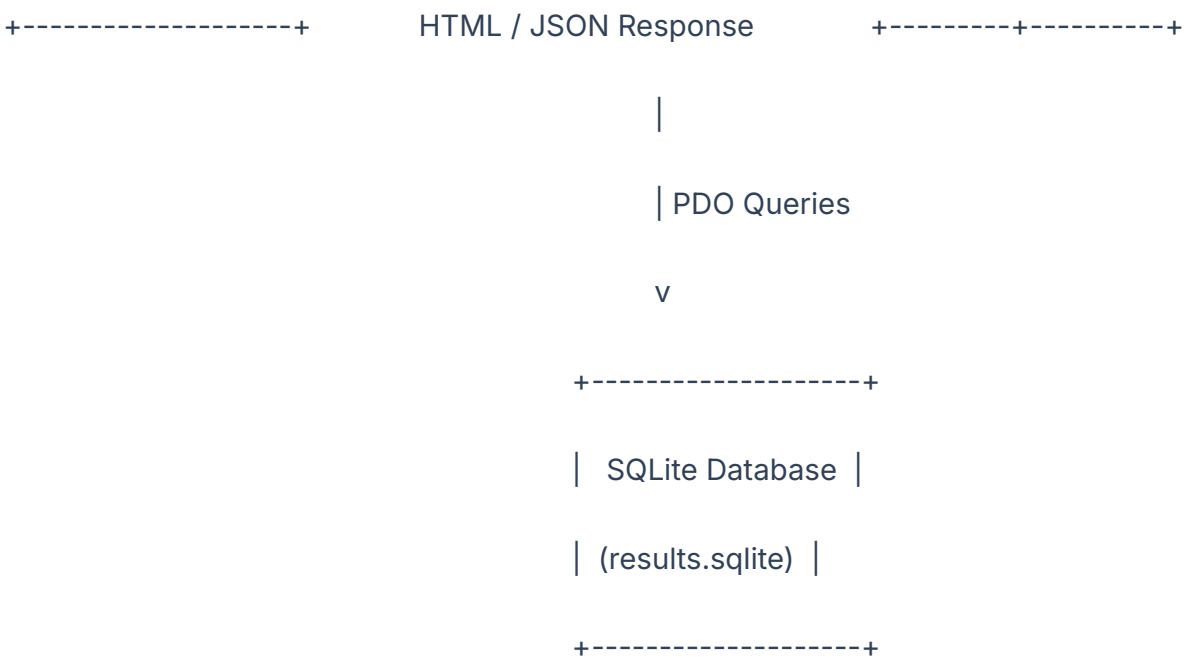
- **Static Pages:** Delivered to the client browser exactly as stored on the file system. Any update requires manual modification of HTML/CSS source files.
- **Dynamic Pages:** Assembled at runtime on the web server. PHP executes scripts, retrieves data from storage/databases, applies business logic (e.g., computing grades and pass rates), and dynamically sends rendered HTML or JSON to the client.

3.2 PHP Architecture and Request-Response Cycle

When a user accesses the system:

1. The browser sends an HTTP request (GET or POST) to the PHP server.
2. The server passes the request to the PHP engine, which parses input via superglobals (`$_GET`, `$_POST`, `$_SERVER`).
3. PHP executes queries via PDO against the SQLite database file (`results.sqlite`).
4. Computed results are returned either as server-rendered HTML or as serialized JSON for asynchronous AJAX requests.





3.3 Database Layer (PDO with SQLite)

- **PDO (PHP Data Objects):** Provides a consistent abstraction layer with parameterized prepared statements, preventing SQL injection vulnerabilities.
- **SQLite Engine:** Serverless, file-based database storing all student records in a dedicated database file, requiring zero daemon setup.
- **Resilient Fallback:** Implemented a transparent fallback to a structured JSON file if the SQLite extension is unavailable in the environment.

4. Procedure and Implementation Steps

1. **Environment Setup:** Configured PHP 8.2 local development server (`php -S 127.0.0.1:8000`) and verified required extensions.
2. **System Configuration (`config.php`):** Configured course definitions (Codes, Titles, Max Marks, Passing thresholds), grading scales, and security helpers (XSS sanitization via `htmlspecialchars` and CSRF generation).
3. **Database Driver (`db.php`):** Implemented a singleton class `DTUDB` to initialize the database table `students`, manage CRUD operations using prepared statements, and enrich raw marks with aggregate totals, percentages, letter grades, GPAs, and backlog detection.

4. **REST API Gateway (`api.php`):** Developed endpoints for `list`, `get`, `create`, `update`, `delete`, and `export_csv` with strict input boundary validations (marks between 0 and 100).
5. **Frontend Interface (`index.php`):** Built the user interface including top navigation, KPI summary cards (Total Students, Class Average, Pass Rate, Class Topper), live search bar, filter tabs, student data table, and modal dialogs.
6. **Interaction and Feedback (`assets/app.js`):** Added instant live calculation of total marks and grades in the enrollment form as user types, keyboard shortcuts for search, asynchronous API calls, and toast alerts.
7. **Official Transcript and Printing:** Built a printable transcript view formatted with course details and institutional seal, supported by a clean print stylesheet.
8. **Testing and Verification:** Tested enrollment of new students, boundary conditions, search and filter responsiveness, transcript generation, record deletion, and CSV download.

5. Source Code Highlights

5.1 System Configuration and Course Registry (`config.php`)

```
<?php

// Session initialization and configuration

if (session_status() === PHP_SESSION_NONE) {

    session_start();

}

define('APP_NAME', 'DTU Result Cloud');

define('INSTITUTION_NAME', 'Delhi Technological University');

define('ACADEMIC_YEAR', '2025 – 2026');

define('SEMESTER_DEFAULT', 'Semester 6');
```

```
// Curriculum registry defining subjects, credits, and passing criteria

define('COURSES', [

    'maths'    => ['code' => 'CS-601', 'title' => 'Advanced Applied Mathematics',      'credits' => 4, 'max' => 100, 'pass' => 40],

    'physics'  => ['code' => 'CS-602', 'title' => 'Quantum & Applied Physics',       'credits' => 3, 'max' => 100, 'pass' => 40],

    'cs'       => ['code' => 'CS-603', 'title' => 'Data Structures & Algorithms',     'credits' => 4, 'max' => 100, 'pass' => 40],

    'web_tech' => ['code' => 'CS-604', 'title' => 'Modern Dynamic Web Architectures', 'credits' => 4, 'max' => 100, 'pass' => 40],

    'english'  => ['code' => 'CS-605', 'title' => 'Technical & Professional Comm.',   'credits' => 2, 'max' => 100, 'pass' => 40]

]);

// Helper for XSS prevention

function h($string): string {

    return htmlspecialchars((string)$string, ENT_QUOTES, 'UTF-8');

}
```

5.2 Database Driver and Academic Enrichment (`db.php`)

```
<?php
class DTUDB {
    private static ?DTUDB $instance = null;
    private $pdo = null;
    private bool $useSqlite = false;
```

```
    private function __construct() {
        $dataDir = __DIR__ . DIRECTORY_SEPARATOR . 'data';
        if (!is_dir($dataDir)) @mkdir($dataDir, 0777, true);
        if (extension_loaded('pdo_sqlite')) {

        try {

            $dbPath = $dataDir . DIRECTORY_SEPARATOR . 'results.sqlite';

            $this->pdo = new PDO("sqlite:" . $dbPath);

            $this->pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

            $this->pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
PDO::FETCH_ASSOC);

            $this->initSqlite();

            $this->useSqlite = true;

        } catch (Exception $e) { $this->useSqlite = false; }

    }

}

public static function getInstance(): DTUDB {

    if (self::$instance === null) self::$instance = new self();

    return self::$instance;

}

private function initSqlite(): void {

    $this->pdo->exec("

        CREATE TABLE IF NOT EXISTS students (
```

```
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            roll_no TEXT UNIQUE NOT NULL,
            name TEXT NOT NULL,
            email TEXT NOT NULL,
            semester TEXT NOT NULL,
            maths INTEGER NOT NULL,
            physics INTEGER NOT NULL,
            cs INTEGER NOT NULL,
            web_tech INTEGER NOT NULL,
            english INTEGER NOT NULL,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    ");
}

// Dynamic academic metrics calculation
public function enrichStudentData(array $student): array {
    $subject
s = [
'Mathematics'          ⇒ (int)$student['maths'],
'Applied Physics'      ⇒ (int)$student['physics'],
'Computer Science'     ⇒ (int)$student['cs'],
```

```
'Advanced Web Tech'    ⇒ (int)$student['web_tech'],
'Professional English' ⇒ (int)$student['english']
];    $total = array_sum($subjects);

   $maxTotal = count($subjects) * 100;

   $percentage = round(($total / $maxTotal) * 100, 1);

   // Grade scale assignment

   if ($percentage >= 90)     { $grade = 'A+'; $gpa = 4.0; $status = 'Distinction';  $color =
'emerald'; }

   elseif ($percentage >= 80) { $grade = 'A';  $gpa = 3.7; $status = 'Distinction';  $color =
'emerald'; }

   elseif ($percentage >= 70) { $grade = 'B';  $gpa = 3.0; $status = 'First Class';   $color
= 'blue'; }

   elseif ($percentage >= 60) { $grade = 'C';  $gpa = 2.5; $status = 'Second Class';
$color = 'indigo'; }

   elseif ($percentage >= 50) { $grade = 'D';  $gpa = 2.0; $status = 'Pass';         $color =
'amber'; }

   else                       { $grade = 'F';  $gpa = 0.0; $status = 'Needs Review'; $color = 'rose';
}

   // Backlog detection (< 40 marks threshold)

   $failed = [];

   foreach ($subjects as $name ⇒ $score) {

      if ($score < 40) $failed[] = $name;

   }
```

```
    $student['total_marks'] = $total;
    $student['max_marks'] = $maxTotal;
    $student['percentage'] = $percentage;
    $student['grade'] = $grade;
    $student['gpa'] = number_format($gpa, 2);
    $student['status'] = count($failed) > 0 ? 'Backlog / Arrear' : $status;
    $student['status_color'] = count($failed) > 0 ? 'rose' : $color;
    $student['failed_subjects'] = $failed;
    $student['subjects_breakdown'] = $subjects;
    return $student;
}

public function getAll(): array {
    $stmt = $this->pdo->query("SELECT * FROM students ORDER BY id DESC");
    return array_map([$this, 'enrichStudentData'], $stmt->fetchAll());
}

public function create(array $d): int {
    $stmt = $this->pdo->prepare("
        INSERT INTO students (roll_no, name, email, semester, maths, physics, cs, web_tech,
english)
        VALUES (:roll_no, :name, :email, :semester, :maths, :physics, :cs, :web_tech, :english)
```

```
    ");

    $stmt->execute([

        ':roll_no' => trim($d['roll_no']), ':name' => trim($d['name']), ':email' =>
trim($d['email']),

        ':semester' => trim($d['semester']), ':maths' => (int)$d['maths'], ':physics' =>
(int)$d['physics'],

        ':cs' => (int)$d['cs'], ':web_tech' => (int)$d['web_tech'], ':english' => (int)$d['english']

    ]);

    return (int)$this->pdo->lastInsertId();

}

public function delete(int $id): bool {

    $stmt = $this->pdo->prepare("DELETE FROM students WHERE id = :id");

    return $stmt->execute
    return $stmt->execute([':id' => $id]);
    }
}
```

5.3 REST API Controller and CSV Exporter (api.php)

```
<?php
require_once __DIR__ . '/config.php';
require_once __DIR__ . '/db.php';
header('Content-Type: application/json; charset=utf-8');

$db = DTUDB::getInstance();
$action = $_GET['action'] ?? $_POST['action'] ?? 'list';
```

```
try {
    switch ($action) {
        case 'list':
            $students = $db->getAll();
            echo json_encode(['success' => true, 'data' => $students]);
            break;

        case 'create':
            $input = json_decode(file_get_contents('php://input'), true) ?: $_POST;
            validateStudentPayload($input);
            $newId = $db->create($input);
            echo json_encode(['success' => true, 'message' => "Student enrolled
successfully.", 'id' => $newId]);
            break;

        case 'delete':
            $id = (int)($_GET['id'] ?? $_POST['id'] ?? 0);
            $deleted = $db->delete($id);
            echo json_encode(['success' => $deleted, 'message' => "Record deleted
successfully."]);
            break;

        case 'export_csv':
            header('Content-Type: text/csv; charset=utf-8');
            header('Content-Disposition: attachment; filename="student_results_' .
date('Ymd_His') . '.csv"');
            $out = fopen('php://output', 'w');
            fputcsv($out, ['ID', 'Roll No', 'Name', 'Email', 'Semester', 'Maths', 'Physics', 'CS',
'Web Tech', 'English', 'Total', 'Percentage', 'Grade', 'GPA', 'Status']);
            foreach ($db->getAll() as $s) {
                fputcsv($out, [$s['id'], $s['roll_no'], $s['name'], $s['email'], $s['semester'],
$s['maths'], $s['physics'], $s['cs'], $s['web_tech'], $s['english'], $s['total_marks'],
$s['percentage'].'%', $s['grade'], $s['gpa'], $s['status']]);
            }
            fclose($out);
            exit;
```

```
        default:
            http_response_code(400);
            echo json_encode(['success' => false, 'error' => 'Invalid action.']);
    }
} catch (Exception $e) {
    http_response_code(400);
    echo json_encode(['success' => false, 'error' => $e->getMessage()]);
}

function validateStudentPayload(array &$p): void {
    if (empty(trim($p['roll_no'] ?? ''))) throw new Exception("Roll Number is required.");
    if (empty(trim($p['name'] ?? '')))    throw new Exception("Student Name is required.");
    if (!filter_var($p['email'] ?? '', FILTER_VALIDATE_EMAIL)) throw new Exception("Valid
email is required.");
    foreach (['maths', 'physics', 'cs', 'web_tech', 'english'] as $sub) {
        $val = (int)($p[$sub] ?? -1);
        if ($val < 0 || $val > 100) throw new Exception("Scores must be between 0 and
100.");
        $p[$sub] = $val;
    }
}
```

5.4 Client-Side Live Calculation and Interactivity (assets/app.js)

```
// Live score calculation & interactive event handling
document.addEventListener('DOMContentLoaded', () => {
    const scoreInputs = ['inputMaths', 'inputPhysics', 'inputCs', 'inputWebTech',
'inputEnglish'];

    // Real-time calculation on input
    scoreInputs.forEach(id => {
        const el = document.getElementById(id);
        if (el) {
            el.addEventListener('input', () => {
                let val = parseInt(el.value, 10);
                if (val > 100) el.value = 100;
```

```
            if (val < 0) el.value = 0;
            updateLiveCalculator();
        });
    }
});

function updateLiveCalculator() {
    let total = 0;
    scoreInputs.forEach(id => {
        const val = parseInt(document.getElementById(id)?.value, 10);
        if (!isNaN(val)) total += val;
    });

    const perc = (total / 500) * 100;
    document.getElementById('previewTotal').textContent = `${total}/500`;
    document.getElementById('previewPerc').textContent = `${perc.toFixed(1)}%`;

    let grade = 'F', color = 'rose';
    if (perc >= 90)      { grade = 'A+'; color = 'emerald'; }
    else if (perc >= 80) { grade = 'A';  color = 'emerald'; }
    else if (perc >= 70) { grade = 'B';  color = 'blue'; }
    else if (perc >= 60) { grade = 'C';  color = 'blue'; }
    else if (perc >= 50) { grade = 'D';  color = 'amber'; }

    const badge = document.getElementById('previewGrade');
    badge.textContent = grade;
    badge.className = `badge badge-${color}`;
}

// Instant search & filter logic
const searchInput = document.getElementById('searchInput');
searchInput?.addEventListener('input', () => {
    const q = searchInput.value.toLowerCase().trim();
    document.querySelectorAll('.student-row').forEach(row => {
        const text = (row.dataset.name + ' ' + row.dataset.roll + ' ' +
row.dataset.email).toLowerCase();
```

```
            row.style.display = text.includes(q) ? '' : 'none';
        });
    });
});
```

6. Screenshots and Output Placeholders

6.1 Dashboard Overview and Student Performance Ledger

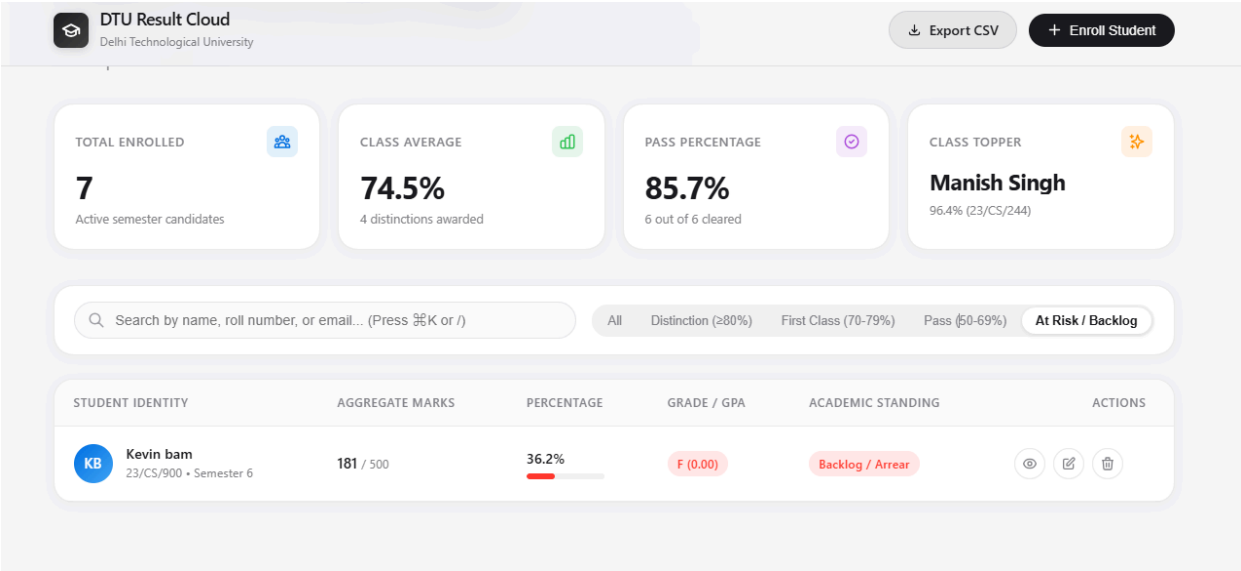


Figure 1: Student Performance Ledger dashboard displaying real-time summary statistics and records table.

6.2 Student Enrollment Modal with Live Score Calculation

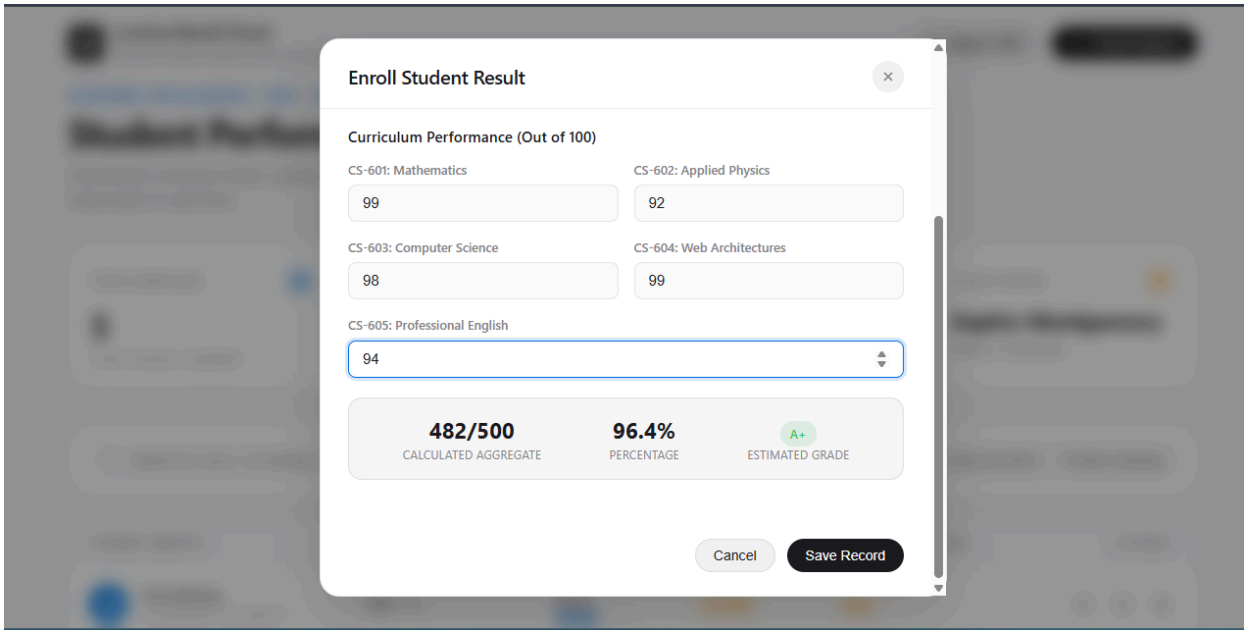


Figure 2: Modal form demonstrating immediate live score and letter grade recalculation as marks are typed.

6.3 Real-Time Search and Segmented Filtering

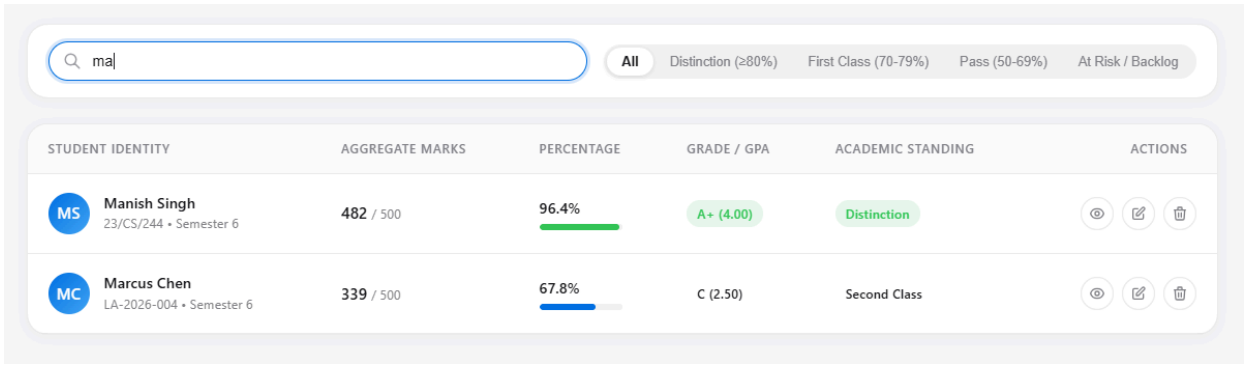


Figure 3: Instant client-side search and category filtering across academic standing tiers.

6.4 Official Printable Academic Transcript

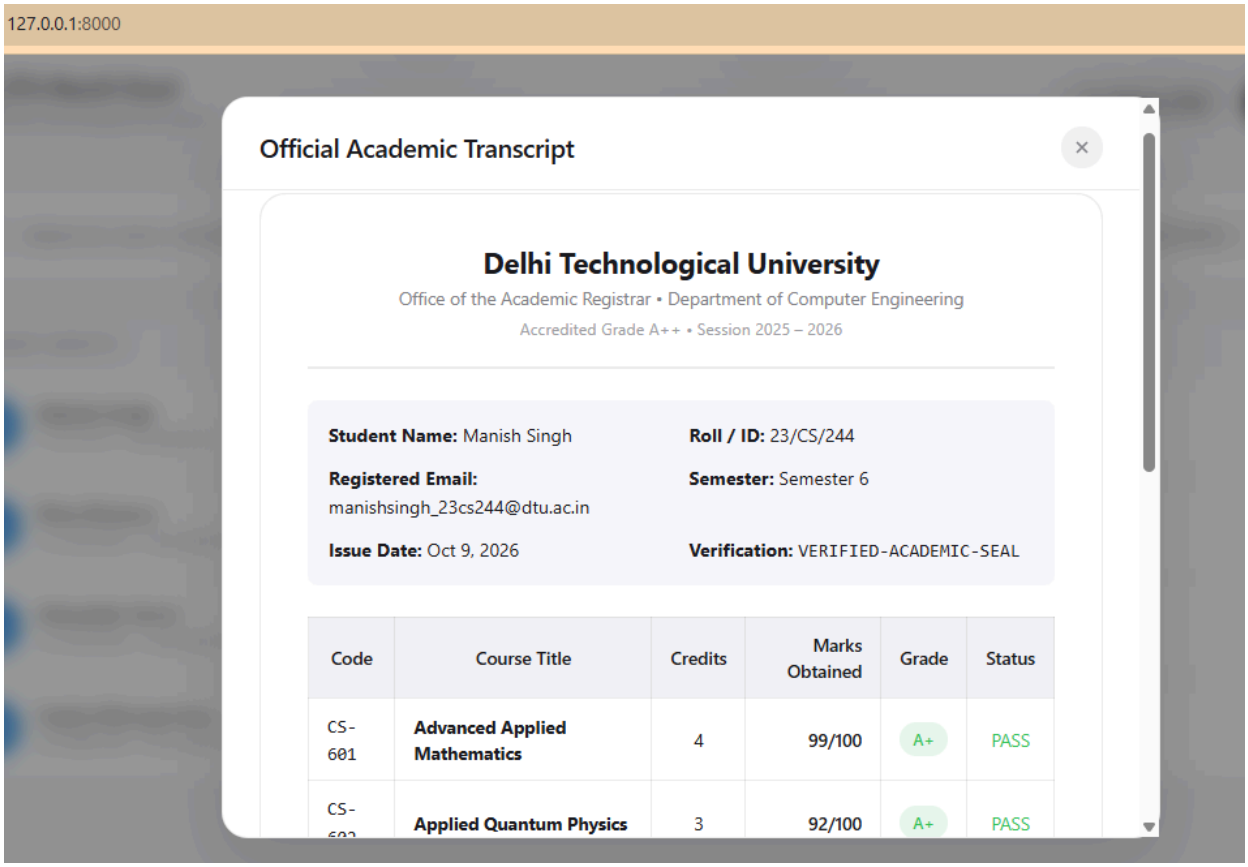


Figure 4: Detailed academic transcript view with verified institutional seal and print option.

6.5 CSV Ledger Export

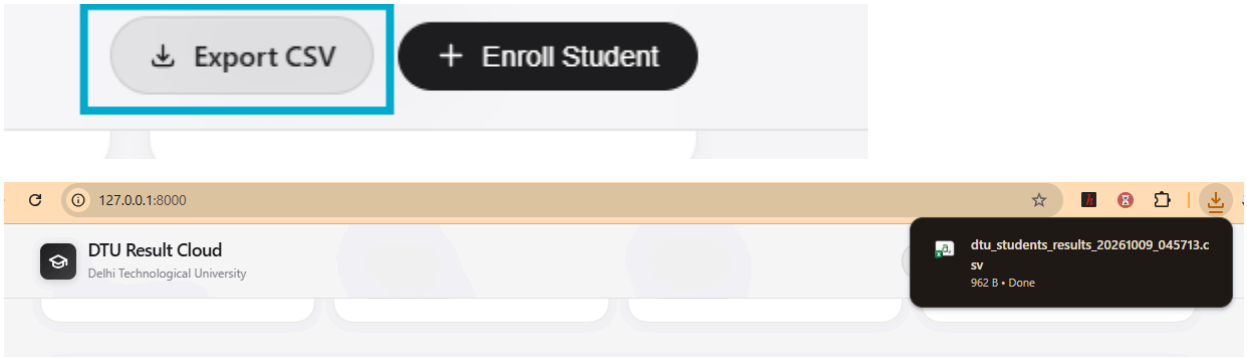


Figure 5: Exported CSV file containing full student result records opened in spreadsheet software.

7. Interaction Design and Usability Implementation

Design Principle	Implementation in Result Management System
Affordances	Distinctly styled text inputs indicating editable fields, clear action buttons with hover states and active press scale feedback.
Signifiers	Search input with search icon and keyboard shortcut badge (⌘K or /), color-coded status badges (green for Distinction, red for Backlog).
Immediate Feedback	Real-time aggregate, percentage, and grade badge updates in the enrollment modal as numbers are entered, along with toast alert banners.
Natural Mapping	Spatial organization of form inputs mirrors standard student mark sheets; table headers directly correspond to report card columns.
Error Prevention	Numeric input boundary constraints (0 to 100), duplicate roll number validation in backend, and explicit browser confirmation dialog before deleting records.

8. Result and Conclusion

The dynamic web application for Student Result Management was successfully developed and verified using PHP and PDO SQLite.
Key results achieved:

- Server-Side Processing: Successfully processed HTTP requests using PHP, organized into modular configuration, database, and API layers.
- Data Persistence: Handled storage reliably using SQLite with parameterized prepared statements ensuring protection against SQL injection.
- Dynamic Calculations: Automated computation of aggregate marks, percentages, letter grades (A+ to F), and GPAs based on academic rubrics.
- Enhanced User Experience: Implemented live arithmetic preview, instant search, multi-tier filtering, toast notifications, and keyboard navigation.

- Report Generation and Portability: Produced an official printable academic transcript with institutional branding and supported CSV data export.

The experiment demonstrated the practical implementation of dynamic web architectures using vanilla PHP, combining robust server-side data management with responsive frontend interactivity.

File Location: D:\7-sem\advWebTechRepo\experiments\exp5\

Key Files: config.php, db.php, api.php, index.php, assets/app.js, assets/style.css

Target Submission PDF: 23_CS_244_Manish_Singh_Exp5_PHP.pdf